Fuzzamoto: Holistic Fuzzing for Bitcoin Protocol Implementations

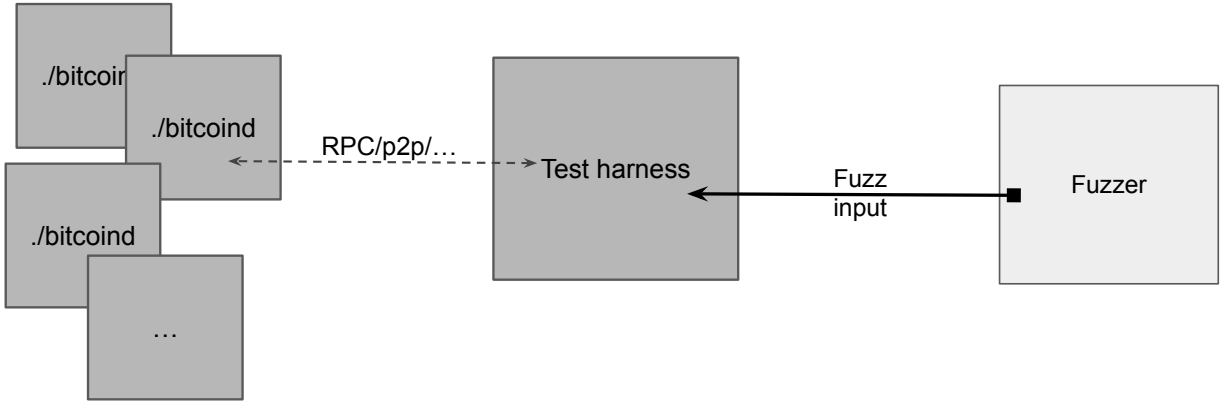
w3st / 19.11.2025

About me

- Niklas Gögge
- Work at Brink (501(c)(3) non-profit funding Bitcoin R&D)
- Mostly fuzzing & other security work on Bitcoin Core

Fuzzamoto

- Fuzzer driven full system simulations of Bitcoin full nodes
  - Testing node daemons through public interfaces
- Testing properties at the system level
- Implementation agnostic (Bitcoin Core, btcd, libbitcoin, …)
- https://github.com/dergoegge/fuzzamoto



Challenge: Non-determinism

- Given the same input, observed coverage should be the same
  - Non-determinism makes fuzzing less efficient (corpus bloat)
  - Non-reproducible bugs are annoying
- What about time? (syscalls, rdtsc instructions, …)
- What about rng? (/dev/urandom, rdrand&rdseed instructions, …)
- How to avoid state accumulation in the targets?
- …

Determinism: Time & RNG

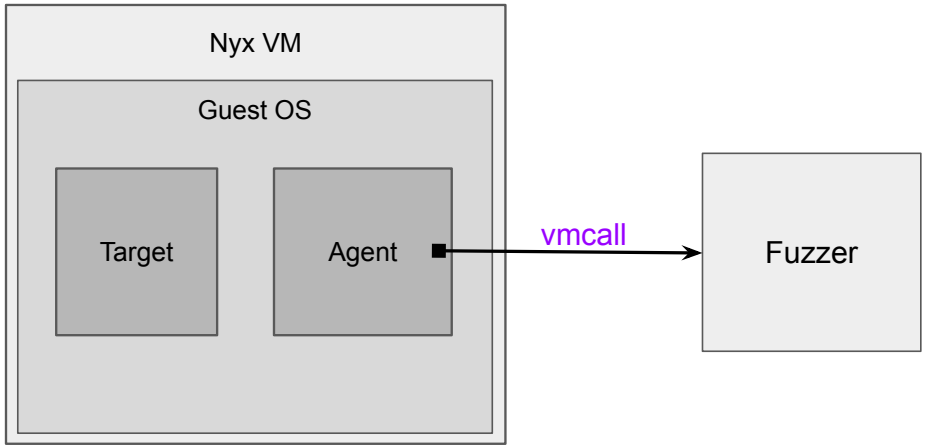
- LD_PRELOAD=libfaketime.so:libfakerand.so ?
  - Doesn’t actually work for various reasons
    - Targets use instructions directly (rdtsc, rdrand)
    - Libraries don’t implement all relevant functions
- (Dumb) workarounds in fuzzamoto for now:
  - Use the mocktime RPC provided by Bitcoin Core
  - Use a patch to nerf RNG
  - Avoid instrumenting uninteresting non-deterministic code
- Still a work in progress
  - Would be nice if the execution environment handled this

Tackling state with Nyx

- Execution backend for full system fuzzing
- Published in 2021 (paper)
- Provides virtual machine with snapshotting capabilities:
  - Create snapshot of whole VM (memory, CPU state, devices, …)
  - Fast reset to snapshot
- Uses KVM to run natively on host CPU (x86 only)
- Snapshotting efficiently reduces non-determinism arising from state
  - I.e. it prevents state from building up in targets across executions
  - “Time travel” to initial state each execution

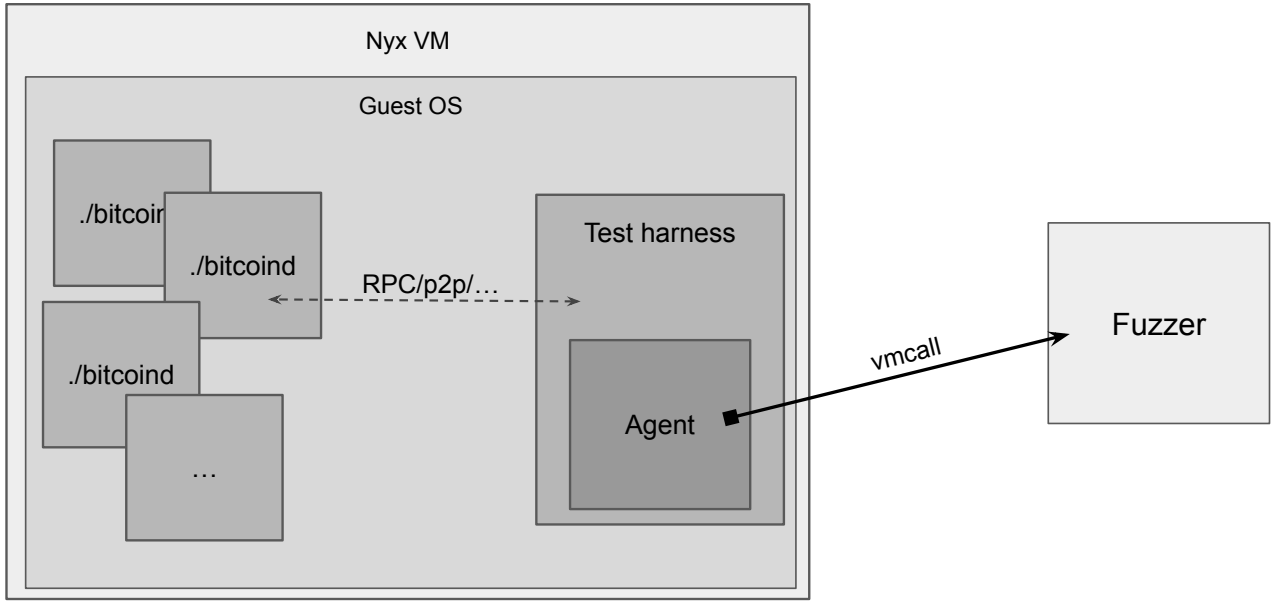
Nyx

- **Agent**
  - Communication between test (inside VM) and fuzzer
  - Wrapper around Nyx's vmcall API
- **Target**: application(s) under test
- vmcall API:
  - "Take snapshot"
  - "Give me the next fuzz input"
  - "Reset VM"
  - "Bug detected"
  - …



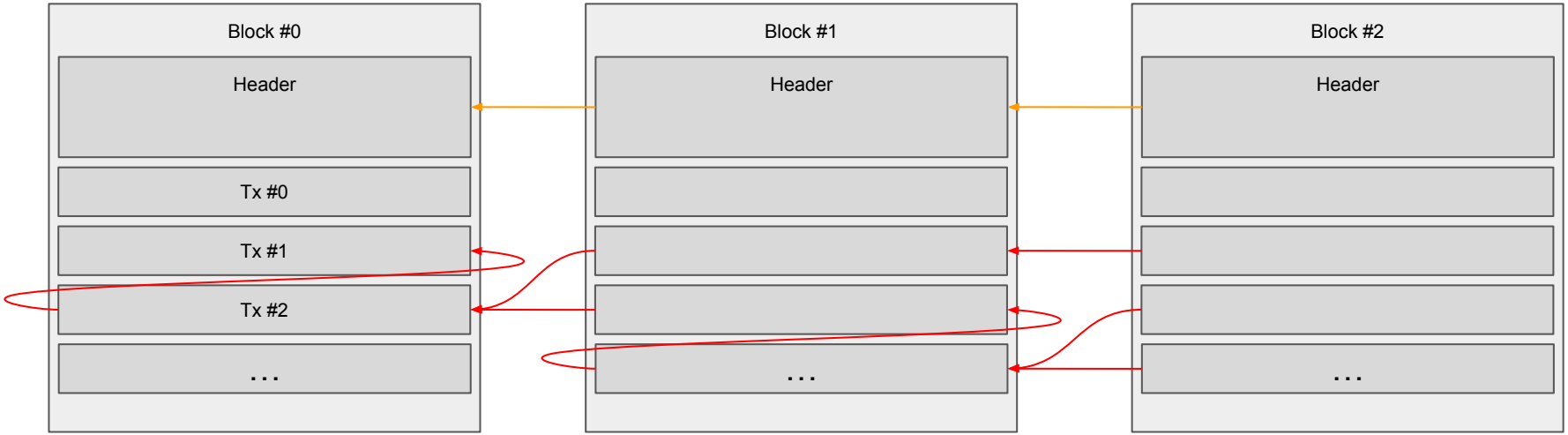
Fuzzamoto with Nyx

1. Test sets up the initial state (spawn nodes, create connection to nodes, mine some blocks, …)
2. Takes snapshot and requests fuzz input
3. Runs the test
4. Resets VM (to state created at 1.)



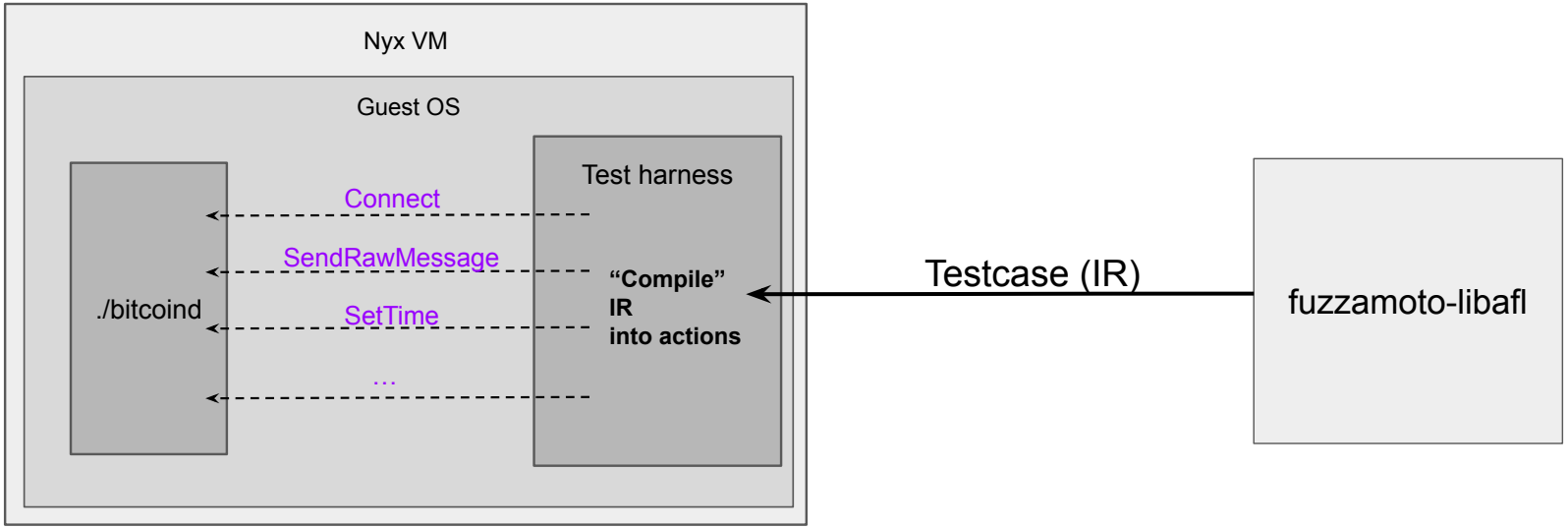
Fuzzing the p2p surface

- Challenge: Bitcoin protocol messages are highly structured, how do we mutate/generate interesting inputs?



Fuzzamoto: Intermediate Representation (IR)

- Test harness executes sequence of three action types
  - “Make a new connection”, “send some bytes”, “change the time”
- IR testcases describe programs for constructing p2p action sequences
- IR is compiled into actions which are then executed



Fuzzamoto: Intermediate Representation (IR)

- Design heavily inspired by Fuzzilli's IL
- Each IR program consists of a sequence of operations
- Operations take some variables as inputs and produce some variables as output
- Testcases may reference data from the snapshot state using Load* operations
  - LoadConnection: Reference an existing connection
  - LoadTxo: Reference a spendable transaction output
  - …
- Load* operations are also used to load data given by the fuzzer into the testcase
  - E.g. LoadBytes, LoadTxVersion, …

```
// Context: nodes=1 connections=8 timestamp=1296688802
v0 <- LoadConnection(5)
v1 <- LoadMsgType("tx")
v2 <- LoadBytes("fefe8520fefefe0000fffffe8520")
SendRawMessage(v0, v1, v2)
```

IR example

- Builds two transactions
- Sends them to the node under test through connection “4” (v2)
- Sends txs in reverse order (v30 then v15)
- Transaction dependencies encoded as variables

```
// Context: nodes=1 connections=8 timestamp=1296688802
v0 <- LoadBytes("5656565656565656567a7a7a7a7a7a7a7a7a
v1 <- LoadTxo(083666c9bf066f9d3a28ad30f5c0ed6fe463f7777
v2 <- LoadConnection(4)
v3 <- LoadTxVersion(2)
v4 <- LoadLockTime(144)
BeginBuildTx(v3, v4) -> v5
  BeginBuildTxInputs -> v6
    v7 <- LoadSequence(4294967295)
    AddTxInput(v6, v1, v7)
  v8 <- EndBuildTxInputs(v6)
  BeginBuildTxOutputs(v8) -> v9
    BeginWitnessStack -> v10
    v11 <- EndWitnessStack(v10)
    v12 <- BuildPayToWitnessScriptHash(v0, v11)
    v13 <- LoadAmount(100000000)
    AddTxOutput(v9, v12, v13)
  v14 <- EndBuildTxOutputs(v9)
v15 <- EndBuildTx(v5, v8, v14)
v16 <- TakeTxo(v15)
v17 <- LoadLockTime(508195987)
v18 <- LoadTxVersion(2)
BeginBuildTx(v18, v17) -> v19
  BeginBuildTxInputs -> v20
    v21 <- LoadSequence(4294967294)
    AddTxInput(v20, v16, v21)
  v22 <- EndBuildTxInputs(v20)
  BeginBuildTxOutputs(v22) -> v23
    v24 <- LoadBytes("51")
    BeginWitnessStack -> v25
    v26 <- EndWitnessStack(v25)
    v27 <- BuildPayToWitnessScriptHash(v24, v26)
    v28 <- LoadAmount(98500000)
    AddTxOutput(v23, v27, v28)
  v29 <- EndBuildTxOutputs(v23)
v30 <- EndBuildTx(v19, v22, v29)
SendTx(v2, v30)
SendTx(v2, v15)
```

IR: Mutators

- **Input** mutator: pick a random operation and change one of its input variables
  - E.g. ‘AddTxInput(v20, v16, v21)‘ => ‘AddTxInput(v20, v1, v21)‘
  - “Change tx/block dependencies”, “change message ordering”, …
- **Operation** mutator: mutate a random operation
  - E.g. ‘LoadTxVersion(2)‘ => ‘LoadTxVersion(3)‘
    - Can be smart about bitcoin specific values (amounts, lock times, …)
  - E.g. ‘LoadBytes(“abababab”)‘ => ‘LoadBytes(“abababac”)‘
- **Splice** mutators: cross-over two existing programs
  - Concat: concat two programs together
  - Combine: insert one program at a random point of another program

IR: Generators

- Mutators operate on existing operations
- Generators insert new operations
  - Helps bootstrapping interesting IR programs
- Various specific generators exist:
  - AdvanceTimeGenerator: generates operations to advance time
  - SingleTxGenerator: generates transaction building operations (single tx)
  - OneParentOneChildGenerator: generates a 1p1c transaction package (two txs)
    - Combination of SingleTxGenerator and input mutations would get there eventually but we know 1p1c is interesting so we help the fuzzer out
  - …

IR Summary

- Describes all/most protocol primitives and their construction
- Leads to combination of protocol primitives in unforeseen & interesting ways

Questions

IR: Minimizers

- Mutators & generators only ever increase IR program size
  => We need minimization to avoid bloat
- Classic strategy (AFL++, libFuzzer):
  “minimize inputs while retaining **all** coverage they trigger”
- Fuzzamoto strategy:
  “minimize inputs while only retaining their **novel** coverage”
- IR Minimizers
  - **Cutting**: attempt to cut of “useless” tail of program (uses binary search)
  - **Block nopping**: Nop whole blocks of operations
  - Single operation **nopping**: Nop single operations

Designing a Nyx Agent for Fuzzamoto

- Default agent (ld_preload_fuzz) does not provide enough control
  => Custom agent necessary
- Leaning on Mozilla's Agent API design
- Provides full control over VM capabilities to the test
  - nyx_init(): Initialize the agent
  - nyx_get_fuzz_input(...): Take the snapshot and get the fuzz input
  - nyx_release(): Reset the VM to the snapshot
  - nyx_fail(...): Inform the fuzzer about a bug (also resets the VM)
  - nyx_skip(), nyx_println(...), nyx_dump_file_to_host(...)

How can we observe coverage?

- Coverage feedback supported by Nyx
  - Intel Processor Trace (not used for Fuzzamoto)
  - Or classic compile-time instrumentation
- Initialization vmcall (nyx_init()) communicates coverage config to fuzzer
  - Coverage map memory address
  - Coverage map size
  - …
- For Fuzzamoto:
  - Bitcoin Core compiled with afl-clang-{fast,lto}
  - nyx_init(): creates shared memory for coverage map
  - nyx_init(): passes reference to shmem to bitcoind through specific afl++ env variables
    => bitcoind (spawned later on) writes coverage to memory created by agent

```
setenv("__AFL_SHM_ID", shmid_str, 1);
setenv("AFL_MAP_SIZE", map_size_str, 1);
```